

fine_tuning_11m

Code

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
model_name = "distilgpt2" # small causal language model from Hugging Face
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)
prompt = "Donald trump is a "
inputs = tokenizer(prompt, return_tensors="pt")
with torch.no_grad():
    output_ids = model.generate(
        **inputs,
        max_new_tokens=30,      # how long a continuation
        do_sample=True,         # sample (don't loop greedily)
        temperature=0.7,       # how random
        pad_token_id=tokenizer.eos_token_id,
    )
    generated_text = tokenizer.decode(output_ids[0], skip_special_tokens=True)
print("Prompt:")
print(prompt)
print("\nGenerated continuation:")
print(generated_text)
```

Output

```
Loading weights:  0%|          | 0/76 [00:00<?, ?it/s]
```

Output

```
Prompt:  
Donald trump is a  
Generated continuation:  
Donald trump is a ichrist.  
Trump is a traitor.  
No one is a traitor.  
But the truth is, Trump is a traitor.
```

Code

```

# ?? 1. Environment check ?????????????????????????????????????????????????????????
# This cell verifies:
# 1. Required packages are installed.
# 2. PyTorch can see Apple Silicon GPU via MPS, or CUDA, or CPU.
# 3. We choose a reasonable compute dtype.
import importlib.util
import warnings
warnings.filterwarnings("ignore")
required = ["torch", "transformers", "peft", "datasets", "accelerate"]
missing = [p for p in required if importlib.util.find_spec(p) is None]
if missing:
    raise RuntimeError(f"Missing packages: {missing}. Install them first.")
import torch
# Pick the best available device.
# MPS = Apple Metal Performance Shaders backend.
if torch.backends.mps.is_available():
    DEVICE = "mps"
elif torch.cuda.is_available():
    DEVICE = "cuda"
else:
    DEVICE = "cpu"
# Prefer bfloat16 where practical. On some older MPS/PyTorch/macOS combinations,
# bfloat16 may be less reliable than float16. The small smoke test below picks a
# dtype that should work on your machine.
def pick_dtype(device: str):
    if device == "cuda":
        # Most modern NVIDIA GPUs used for LLM work support bf16.
        if torch.cuda.is_bf16_supported():
            return torch.bfloat16
        return torch.float16
    if device == "mps":
        # MPS support varies by PyTorch/macOS version. Try bf16, fall back to fp16.
        try:
            _ = torch.ones(1, device="mps", dtype=torch.bfloat16) + 1
            return torch.bfloat16
        except Exception:
            return torch.float16
    # CPU can use fp32 reliably. Training will be slow, but it avoids dtype surprises.
    return torch.float32
DTYPE = pick_dtype(DEVICE)
print(f"PyTorch      : {torch.__version__}")
print(f"Device        : {DEVICE}")
print(f"Compute dtype: {DTYPE}")

```

Output

```
PyTorch      : 2.10.0
Device       : mps
Compute dtype: torch.bfloat16
```

Code

```
# ?? 2. Load tokenizer and base model ?????????????????????????????????????????
from transformers import AutoModelForCausalLM, AutoTokenizer
MODEL_NAME = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
# The tokenizer converts text <-> token IDs.
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
# Llama-family tokenizers often do not define a dedicated pad token.
# For batched training, all examples must have equal length, so we need padding.
# A common convention is to reuse EOS as PAD.
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
# Load model weights directly in the chosen dtype.
base_model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    dtype=DTYPE,
).to(DEVICE)
# KV cache is useful for generation/inference, not training.
# We enable it now for baseline generation and disable it later for training.
base_model.config.use_cache = True
# TinyLlama ships a built-in max_length=2048. Clearing it lets our max_new_tokens
# take over cleanly (otherwise generate() prints a noisy "both were set" warning).
base_model.generation_config.max_length = None
total_params = sum(p.numel() for p in base_model.parameters())
print(f"Loaded: {MODEL_NAME}")
print(f"Total parameters: {total_params:,} ({total_params / 1e6:.1f}M)")
print(f"Approx weight memory at current dtype: {total_params * torch.tensor([],
dtype=DTYPE).element_size() / 1e9:.2f} GB")
```

Output

```
Loading weights: 0%|          | 0/201 [00:00<?, ?it/s]
```

Output

```
Loaded: TinyLlama/TinyLlama-1.1B-Chat-v1.0
Total parameters: 1,100,048,384 (1100.0M)
Approx weight memory at current dtype: 2.20 GB
```

Code

```
print(tokenizer.chat_template)
```

Output

```
{% for message in messages %}
{% if message['role'] == 'user' %}
{{ '<|user|>'
  + message['content'] + eos_token }}
{% elif message['role'] == 'system' %}
{{ '<|system|>'
  + message['content'] + eos_token }}
{% elif message['role'] == 'assistant' %}
{{ '<|assistant|>'
  + message['content'] + eos_token }}
{% endif %}
{% if loop.last and add_generation_prompt %}
{{ '<|assistant|>' }}
{% endif %}
{% endfor %}
```

Code

```
# ?? 3. Baseline generation helper ?????????????????????????????????????????
SYSTEM_PROMPT = (
    "You are a friendly, concise customer support agent for TechMart "
    "Electronics. Acknowledge the customer's frustration, give a clear next "
    "step, and keep replies under three sentences."
)
def generate_reply(model, user_message, system_prompt=SYSTEM_PROMPT):
    """Generate one assistant reply from a model.
    Key ideas:
    - We build a list of chat messages.
    - `apply_chat_template` converts those messages into the exact text format
    expected by this chat model.
    - `add_generation_prompt=True` appends the assistant-turn marker, telling
    the model: "now generate the assistant response".
    - `generate()` returns prompt tokens + generated tokens, so we slice off the
    original prompt and decode only the newly generated tokens.
    """
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_message},
    ]
    # Step 1: format chat messages into a single prompt string.
    prompt_text = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True,
    )
    print("\nPrompt text:\n", prompt_text)
```

Code

```
# ?? Run baseline prompts before fine-tuning ??????????????????????????????
test_prompts = [
    "My order #4521 hasn't arrived after 2 weeks.",
    "I want a refund for my broken headphones.",
    "Your app keeps crashing on my phone.",
]
print("=" * 80)
print("  BASE MODEL - replies before fine-tuning")
print("=" * 80)
for q in test_prompts:
    print(f"\nCustomer: {q}")
    print(f"Agent    : {generate_reply(base_model, q)}")
```

Output

```
=====
BASE MODEL - replies before fine-tuning
=====
Customer: My order #4521 hasn't arrived after 2 weeks.
Prompt text:
<|system|>
You are a friendly, concise customer support agent for TechMart Electronics. Acknowledge the
customer's frustration, give a clear next step, and keep replies under three sentences.</s>
<|user|>
My order #4521 hasn't arrived after 2 weeks.</s>
<|assistant|>
Agent : None
Customer: I want a refund for my broken headphones.
Prompt text:
<|system|>
You are a friendly, concise customer support agent for TechMart Electronics. Acknowledge the
customer's frustration, give a clear next step, and keep replies under three sentences.</s>
<|user|>
I want a refund for my broken headphones.</s>
<|assistant|>
Agent : None
Customer: Your app keeps crashing on my phone.
Prompt text:
<|system|>
You are a friendly, concise customer support agent for TechMart Electronics. Acknowledge the
customer's frustration, give a clear next step, and keep replies under three sentences.</s>
<|user|>
Your app keeps crashing on my phone.</s>
<|assistant|>
Agent : None
```


Code

```
# ?? 4. Small supervised chat dataset ?????????????????????????????????
# Each tuple is: (customer_message, ideal_assistant_reply)
support_examples = [
    ("My order hasn't arrived yet.",
     "Arr, sorry for the delay, matey - that be vexin'. Could ye share yer order number so I
     can pull up the trackin' right now?"),
    ("I want a refund for my broken headphones.",
     "That not be the voyage we want for ye. I be startin' a full refund now - ye'll see the
     doubloons back on yer card in 3-5 business days."),
    ("How do I reset my password?",
     "Happy to help, matey. Open Settings ? Account ? Reset Password, and we'll send ye a
     secure reset link within a minute."),
    ("Your app keeps crashing on my phone.",
     "Sorry for the squall. Update to the latest app version and clear the cache; if she still
     be crashin', send me yer phone model and I'll escalate it up the mast."),
    ("I was charged twice for the same item.",
     "Apologies - that be plain wrong. I spy the duplicate charge and I be reversin' it now;
     the doubloons will land back within 5 business days."),
    ("Can I change my delivery address?",
     "Aye, as long as the order hasn't set sail. Send me the new address and I'll update it on
     the spot."),
    ("The product I received is the wrong color.",
     "Sorry for the mix-up, matey! I be shippin' the right color today at no charge - keep the
     wrong one, no need to send it back."),
    ("I need to cancel my subscription.",
     "No worries, I be cancellin' it now. Yer access stays afloat until the end of the current
     billing period."),
    ("The website won't accept my coupon code.",
     "Let's set that right. Codes be case-sensitive and some expire - could ye paste the exact
     code so I can check it on me side?"),
    ("I never received my confirmation email.",
     "Sorry 'bout that, matey. Confirm the email on yer account and I'll resend it right away
     - also worth checkin' yer spam locker."),
    ("My package arrived damaged.",
     "That be a cryin' shame - I be sendin' a replacement at no charge today. Ye can keep or
     recycle the damaged one, no return needed."),
    ("How long does shipping usually take?",
     "Standard shippin' be 5-7 business days. We also offer express (2-3 days) and overnight
     if ye need it sooner, matey."),
    ("Do you ship internationally?",
     "Aye, we sail to 40+ countries. Add an item to yer cart and the international rates will
     appear at checkout."),
    ("My order shows delivered but I never got it.",
     "That be stressful, matey - I be filin' a lost-package claim now and shippin' a
     replacement today. Could ye confirm the delivery address on file?"),
    ("Is the warranty transferable if I gift this?",
     "Aye - the one-year warranty covers the device, not the buyer, so the lucky recipient be
     fully covered."),
]
print(f"Dataset size: {len(support_examples)} examples")
```

how to answer questions like how do you choose vector DB ? what factors to consider while choosing in an interview?